

Market Basket Analysis EDA Report

Dataset:

In order for apriori and fpgrowth algorithms to work, I have created a sample of first 50k rows.

GSP algorithm works on the full dataset.

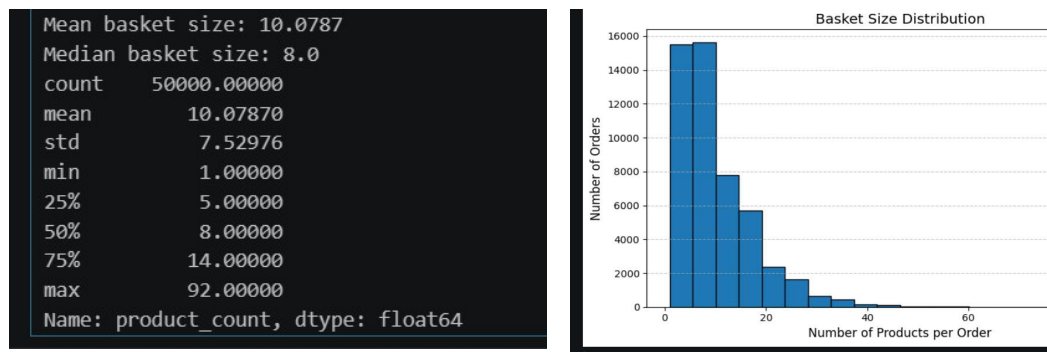
```
raw_df = pd.read_csv("dataset_son.csv")

# df = pd.read_csv("dataset_son.csv")

df = raw_df.head(50000)
df.to_csv("sample_50000_rows.csv", index=False)
```

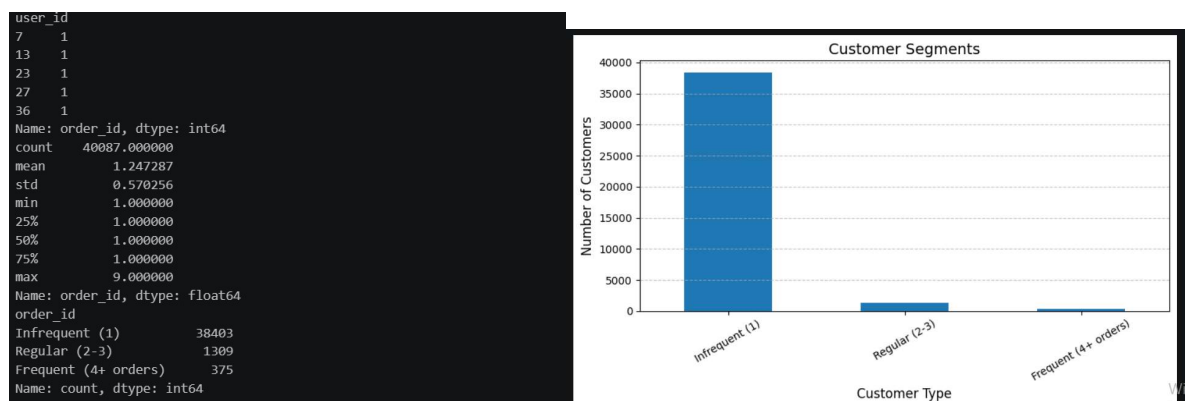
Basket Analysis:

We have an average of 10 items per order, while median is 8 and std is 7. Most customers purchased 10 or lower items however some with large purchasing customers also do exist.



Customer Frequency:

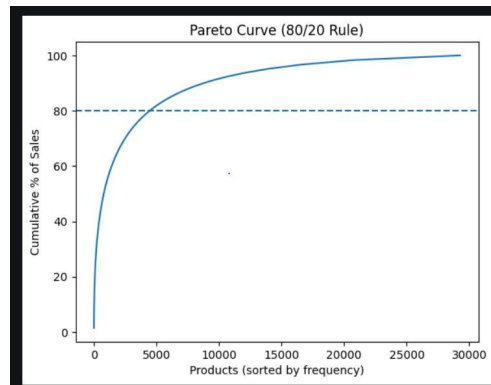
More than %95 of the customers only bought once some regular buyers also exist but few in number.



Pareto Analysis:

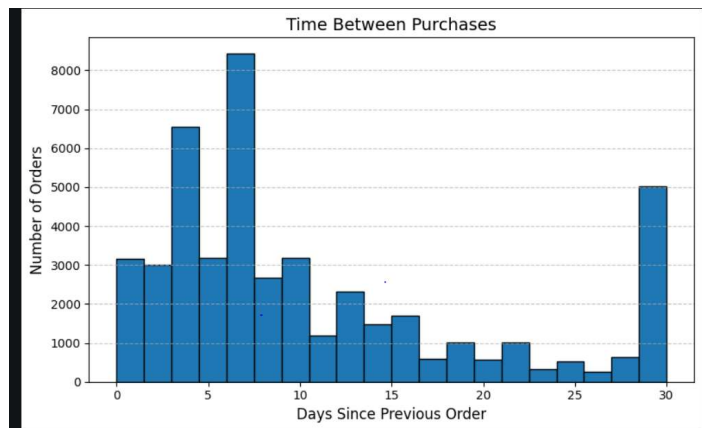
As we can see, a few certain products dominate customer purchases, and will be very useful later on in our recommendation algorithms. If necessary to decrease ram need, we can lower item numbers to %15 of all items since we can observe they make %80 of the sales already.

```
product_list
24852 7422
13176 5965
21137 4059
21903 3822
47209 3342
Name: count, dtype: int64
4461 out of 29267 products account for 80% of sales
That is 15.24% of all products
```



Time Between Purchases:

The graph below shows many customers did make a second purchase within first 15 days (peaks between 5-10 day interval). 30 or above days also account for a good portion of the customers.



Basket Matrix One-Hot Encoding:

Here we have the code to convert our order sets to basket matrix, in the end this matrix contains mostly zeroes as it is a sparse matrix. With the initial dataset we had a ram bottleneck in this step.

```
df["product_list"] = df["product_id"].apply(
    lambda x: [i.strip() for i in str(x).split(",") if i.strip()]
)

transactions = df.explode("product_list")

transactions = transactions[["order_id", "product_list"]]

basket = (transactions
          .groupby(["order_id", "product_list"])["product_list"]
          .count()
          .unstack()
          .fillna(0))

basket = (basket > 0).astype(int)

print(basket.head()) # all zeroes - sparse matrix
✓ 2m 1.7s
```

Association Mining

Apriori Analysis:

From mlxtend we import the apriori algorithm, also keep track of time and memory for a comparison with FP-growth.

After fine tuning, we choose support = 0.05, confidence = 0.15, lift = 1. We could have gone lower to increase our rule number in this case is only 75. However apriori kept taking way more time as we can lowered our parameters.

We sort the rules we find by support- confidence and lift here it shows top 5 rules:

```
Top 5 Rules by Support
antecedents consequents support confidence lift
10 47209 13176 0.01920 0.287253 2.407822
11 13176 47209 0.01920 0.160939 2.407822
55 47766 24852 0.01676 0.307862 2.073982
2 27966 13176 0.01340 0.305378 2.559751
24 21137 47209 0.01270 0.156442 2.340552

Top 5 Rules by Confidence
antecedents consequents support confidence lift
48 28204 24852 0.01028 0.361972 2.438506
50 45066 24852 0.00918 0.358034 2.411980
58 49683 24852 0.01012 0.331152 2.230880
1 19057 13176 0.00754 0.325843 2.731288
55 47766 24852 0.01676 0.307862 2.073982

Top 5 Rules by Lift
...
68 47626 26209 0.00812 0.171308 4.021315
69 26209 47626 0.00812 0.190610 4.021315
Apriori memory usage (MB): -4633.953125
Apriori runtime: 470.58409309387207
```

FP-Growth:

Here is the output of our fpgrowth algorithm, as we can see we received the same outputs as apriori, but with way less time, due to its tree structure, below is a more detailed comparison between them.

```
FP-Growth - Top 5 Rules by SUPPORT
antecedents consequents support confidence lift
6 47209 13176 0.01920 0.287253 2.407822
7 13176 47209 0.01920 0.160939 2.407822
26 21137 13176 0.01874 0.230845 1.934996
27 13176 21137 0.01874 0.157083 1.934996
24 21137 24852 0.01834 0.225918 1.521946

FP-Growth - Top 5 Rules by CONFIDENCE
antecedents consequents support confidence lift
76 28204 24852 0.01028 0.361972 2.438506
46 45066 24852 0.00918 0.358034 2.411980
70 49683 24852 0.01012 0.331152 2.230880
121 19057 13176 0.00754 0.325843 2.731288
29 47766 24852 0.01676 0.307862 2.073982

FP-Growth - Top 5 Rules by LIFT
...
132 47626 26209 0.00812 0.171308 4.021315
133 26209 47626 0.00812 0.190610 4.021315
FP-Growth memory usage (MB): 9626.25390625
FP-Growth runtime: 76.55665016174316
```

Apriori and FP-Growth Comparison:

FP growth gave the same performance apriori when we compare lift, support and confidence. FP-growth's tree structure compresses the data while apriori generated itemsets separately, which will cause memory and time differences.

Time:

Firstly with 5k order data apriori took 1.05 s and fpgrowth 0.4 s, however FPgrowth actually took more memory with 3.1mb vs 1mb on apriori which is an interesting observation.

50k rows sample data: fp growth took 1.16 minutes while apriori took 7.30 minutes. When we increased the parameters by doubling support lift and confidence, FPgrowth has taken 10.5s whereas apriori took 24s to run. We observe an exponential increase in apriori when we increase data size and lower the parameters, which makes it very hard to implement on large datasets.

Memory:

FP-growth with our 50k dataset took 9.6GB memory, whereas apriori algorithm took 5.6 GB Memory, when we doubled the parameters, thereby lowering the rules we look for, FP-growth took 196MB while apriori took 560MB.

Normally we would expect FP-growth to take less memory as the dataset size increases due to its tree-structure, here we have an anomaly, however it is probably case-specific.

Conclusion:

On large datasets, we observe FP-growth gives almost same output and recognizes patterns with similar ability while being 3 times faster, however it has taken more memory in our application.

Sequential Rule Mining

We go back to our original dataset here with df_full.

Firstly we will try without constraints with pymining library:

```
df_full = pd.read_csv("dataset_son.csv")
df_full = df_full[[
    "user_id",
    "order_id",
    "product_id",
    "days_since_prior_order"
]]

df_full = df_full.sort_values(by=["user_id", "order_id"])

df_full["product_list"] = df_full["product_id"].apply(
    lambda x: [i.strip() for i in str(x).split(",") if i.strip()]
)

df_full["product_list"] = df_full["product_id"].apply(
    lambda x: tuple(i.strip() for i in str(x).split(",") if i.strip())
)

user_sequences = df_full.groupby("user_id")["product_list"].apply(list)
sequences = user_sequences.tolist()

print(sequences[:2])
```

```
# after 45 min didn't stop running
# from pymining import seqmining

# min_support = 2

# freq_seqs = seqmining.freq_seq_enum(sequences, min_support)

# print(list(freq_seqs)[:10])
```

Problem:

Without constraints for max gap this GSP algorithm runs forever since it has an exponential runtime. After 45 minutes I gave up and continued with min max gap constraints. So we cannot use pymining library at all.

GSP:

Below we implement the algorithm and append the rules we found to patterns_full list, also the counter number showing how many times we have observed that pattern.

```
from collections import Counter

df_full["days_since_prior_order"] = df_full["days_since_prior_order"].fillna(0)

df_full["cumulative_days"] = df_full.groupby("user_id")["days_since_prior_order"].cumsum()

user_sequences_time = df_full.groupby("user_id").apply(
    lambda x: list(zip(x["product_list"], x["cumulative_days"]))
)

print(user_sequences_time.iloc[0])

def run_gsp_full(user_sequences_time, min_gap=0, max_gap=15, min_support=3):

    def gsp_pairs(sequence):
        pairs = []

        for i in range(len(sequence)):
            items_i, time_i = sequence[i]

            for j in range(i + 1, len(sequence)):
                items_j, time_j = sequence[j]

                gap = time_j - time_i

                if gap < min_gap:
                    continue
                if gap > max_gap:
                    break

                pairs.append((items_i, items_j))

        return pairs
```

Sample output:

```
[(('196', '12427', '10258', '25133', '10326', '17122', '41787', '13176'), 28.0),
```

(This sequential order set is observed for a total of 28 times.)

Filtering Patterns:

Below we filter patterns so that an item A gets linked to a single item B to increase its interpretebility. We still keep patterns_full but we will use both of them.

```
def filter_patterns(patterns_full, mode="simple"):

    if mode == "simple":
        # A → B
        return {
            p: c for p, c in patterns_full.items()
            if len(p[0]) == 1 and len(p[1]) == 1 and p[0] != p[1]
        }

    elif mode == "medium":
        # allow small groups
        return {
            p: c for p, c in patterns_full.items()
            if len(p[0]) <= 2 and len(p[1]) <= 2
        }

    else:
        # return full
        return patterns_full
```

Experimentation:

Here is GSP applied with max gap = 5, 15, 25

```
max_gaps = [5, 15, 25]

for gap in max_gaps:
    print(f"\n=====")
    print(f"MAX GAP = {gap}")
    print(f"=====")

    patterns_full = run_gsp_full(user_sequences_time, max_gap=gap, min_support=3)

    print("\nTop Full patterns:")
    print(Counter(patterns_full).most_common(5))

    patterns_simple = filter_patterns(patterns_full, mode="simple")

    print("\nTop Simple patterns (A → B):")
    print(Counter(patterns_simple).most_common(5))
```

Output:

```
MAX GAP = 5
=====
Top Full patterns:
[(((('46822',), ('46822',)), 249), (((('24852',), ('24852',)), 237), (((('19660',), ('19660',)), 173), (((('196',), ('196',)), 168), (((('40939',),
Top Simple patterns (A → B):
[(((('33249',), ('17902',)), 53), (((('17902',), ('33249',)), 48), (((('1463',), ('19677',)), 34), (((('19677',), ('1463',)), 24), (((('27729',), ('
=====
MAX GAP = 15
=====
Top Full patterns:
[(((('46822',), ('46822',)), 721), (((('24852',), ('24852',)), 672), (((('19660',), ('19660',)), 561), (((('196',), ('196',)), 558), (((('38928',),
Top Simple patterns (A → B):
[(((('33249',), ('17902',)), 130), (((('17902',), ('33249',)), 128), (((('1463',), ('19677',)), 62), (((('19677',), ('1463',)), 61), (((('23288',),
=====
MAX GAP = 25
=====
...
[(((('46822',), ('46822',)), 1064), (((('24852',), ('24852',)), 1014), (((('19660',), ('19660',)), 997), (((('196',), ('196',)), 963), (((('38928',),
Top Simple patterns (A → B):
[(((('17902',), ('33249',)), 193), (((('33249',), ('17902',)), 189), (((('1463',), ('19677',)), 102), (((('19677',), ('1463',)), 91), (((('23288',),
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Analysis:

Full patterns mostly show item A to item A pattern, meaning customers are more likely to buy the same item in their next purchase. This is obviously not much of any help to us.

Simple pattern filtering greatly helps us in this scenario as we can observe actual recommendations. The most purchased items in sequence are 17902 and 33249, with almost 200 instances.

While increasing max_gap we do not see any meaningful changes in patterns, only an increase in the times they have been observed.

Recommendation System

Unknown User (FP Growth):

On the left side is our function for recommendation to an unknown user. With the rules_model array we have copied previously from FP_growth algorithm that stores rules, we apply the algorithm for certain baskets and on the right side shows the recommendations our algorithm gives:

```
def recommend_unknown_user(rules_model, basket, top_n=5):
    basket_set = set(basket)
    recommendations = []

    for _, row in rules_model.iterrows():
        antecedents = set(row["antecedents"])
        consequents = set(row["consequents"])

        if antecedents.issubset(basket_set):
            for item in consequents:
                if item not in basket_set:
                    recommendations.append((item, row["lift"], row["confidence"]))

    recommendations = sorted(recommendations, key=lambda x: (-x[1], -x[2]))

    seen = set()
    final = []
    for item, _, _ in recommendations:
        if item not in seen:
            seen.add(item)
            final.append(item)

    return final[:top_n]
```

Recommendation Samples

```
basket = ["13176"]
print("Basket:", basket)
print("Recommendations:", recommend_unknown_user(rules_model, basket))

basket = ["21903"]
print("Basket:", basket)
print("Recommendations:", recommend_unknown_user(rules_model, basket))
basket = ["45007"]
print("Basket:", basket)
print("Recommendations:", recommend_unknown_user(rules_model, basket))
```

0.0s

```
Basket: ['13176']
Recommendations: ['47209', '21137']
Basket: ['21903']
Recommendations: ['21137', '13176', '24852']
Basket: ['45007']
Recommendations: ['21903', '47209', '21137', '13176', '24852']
```

Known User (GSP):

We used GSP to recommend for known users as it uses sequential (past) data. We use both user_id and current basket in our function. Below shows two examples of known user recommendation:

```
def recommend_known_user(user_id, user_sequences_time, patterns_full, basket, top_n=5):
    if user_id not in user_sequences_time.index:
        return []

    basket_set = set(basket)

    user_history = user_sequences_time.loc[user_id]
    historical_itemsets = [set(items) for items, _ in user_history]

    recommendations = []

    for (prev, next_), support in patterns_full.items():
        prev_set = set(prev)
        next_set = set(next_)

        basket_match = prev_set.issubset(basket_set)

        history_match = any(prev_set.issubset(hist_set) for hist_set in historical_itemsets)

        if basket_match or history_match:
            for item in next_set:
                if item not in basket_set:
                    recommendations.append((item, support))

    recommendations.sort(key=lambda x: -x[1])

    final_recommendations = []
    seen = set()

    for item, _ in recommendations:
        if item not in seen:
            seen.add(item)
            final_recommendations.append(item)

    return final_recommendations[:top_n]
```

```
patterns_simple = {
    p: c for p, c in patterns_full.items()
    if len(p[0]) == 1 and len(p[1]) == 1 and p[0] != p[1]
}
```

```
# Example 1

basket = ["33120", 28985, 9327, 45918, 30035, 17794, 40141, 1819, 43668"]

recs = recommend_known_user(
    user_id=2,
    user_sequences_time=user_sequences_time,
    patterns_full=patterns_simple,
    basket=basket,
    top_n=5
)

print("Recommendations: ", recs)

# Example 2

basket = ["13176", 15005, 47329, 27966, 23909, 48370, 13245, 9633, 27360, 6348, 40878, 6184, 48002, 20914, 37011, 12962]

recs = recommend_known_user(
    user_id=5,
    user_sequences_time=user_sequences_time,
    patterns_full=patterns_simple,
    basket=basket,
    top_n=5
)

print("Recommendations:", recs)
```

0.0s

```
Recommendations: ['19660', '9387', '43352', '16797', '6184']
Recommendations: ['39275', '41544', '30638', '11198', '12015']
```

Windows'u Etkinle...

END OF REPORT